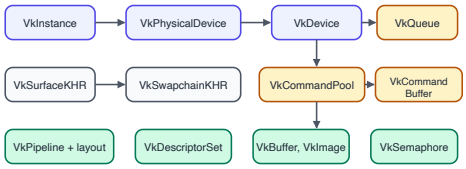
Vulkan Cheat Sheet

Vulkan 1.4 · SDK 1.4.363.0 · Roadmap 2026 · checked 1 November 2026 · Irurueta Docs, Graphics & Games

Version baseline (November 2026)

Item	Status
Core	**Vulkan 1.4** (3 Dec 2024); headers **1.4.367** (30 Oct 2026); no 1.5 announced
SDK	LunarG **1.4.363.0** (29 Sep 2026), Windows, Linux, macOS; `vkprofiles` tool
Roadmap	Roadmap 2022 / 2024 / **2026** (announced 23 Jan 2026) with `VK_EXT_descriptor_heap`
Tools	Slang v2026.20, VMA 3.5.0, Vulkan-Hpp in the SDK
Apple	MoltenVK 1.4.2; **KosmicKrisp** 1.4 conformant, Metal 4, macOS 26+
Android	Android 7+; 1.1 on 64-bit Android 10+; Android 16 = 1.4
Safety	Vulkan SC 1.0.22 (based on 1.2)

Object hierarchy



Instance → physical device → device → queue, command pool → command buffer. One pool per thread per frame.

Minimal frame loop

1. Wait for frame *N-2*: timeline value (`vkWaitSemaphores`) or fence
2. `vkAcquireNextImageKHR` signals the acquire semaphore; rebuild the swapchain on `OUT_OF_DATE`
3. Reset the pool; record: `vkCmdPipelineBarrier2` (`UNDEFINED` → `COLOR_ATTACHMENT_OPTIMAL`), `vkCmdBeginRendering`, bind pipeline and sets, draw, `vkCmdEndRendering`, barrier2 (→ `PRESENT_SRC_KHR`)
4. `vkQueueSubmit2`: wait acquire; signal render-done and the timeline value
5. `vkQueuePresentKHR` waits render-done; advance the frame index (2 or 3 frames in flight)

Image layout transitions and sync2 quick map

Use	Old → new layout	src stage / access → dst stage / access
Upload	`UNDEFINED` → `TRANSFER_DST_OPTIMAL`	`NONE` → `COPY` / `TRANSFER_WRITE`
Sample	`TRANSFER_DST_OPTIMAL` → `SHADER_READ_ONLY_OPTIMAL`	`COPY` / `TRANSFER_WRITE` → `FRAGMENT_SHADER` / `SHADER_SAMPLED_READ`
Draw	`UNDEFINED` → `COLOR_ATTACHMENT_OPTIMAL`	`COLOR_ATTACHMENT_OUTPUT` / `COLOR_ATTACHMENT_WRITE`
Depth	`UNDEFINED` → `DEPTH_ATTACHMENT_OPTIMAL`	`EARLY_` + `LATE_FRAGMENT_TESTS` / `DEPTH_STENCIL_ATTACHMENT_WRITE`
Present	`COLOR_ATTACHMENT_OPTIMAL` → `PRESENT_SRC_KHR`	`COLOR_ATTACHMENT_OUTPUT` / `WRITE` → `NONE`
Compute → draw	`GENERAL` (storage)	`COMPUTE_SHADER` / `SHADER_STORAGE_WRITE` → `VERTEX_SHADER` / `SHADER_STORAGE_READ`

Names are `VK_PIPELINE_STAGE_2_*_BIT` and `VK_ACCESS_2_*_BIT`. Name the exact stage; avoid `ALL_COMMANDS` in shipping code. Queue-family transfer: release on the source, acquire on the destination.

Memory type selection and VMA flags

Need	Memory property flags	VMA
GPU-only image or buffer	`DEVICE_LOCAL`	`VMA_MEMORY_USAGE_AUTO`
Staging upload	`HOST_VISIBLE` + `HOST_COHERENT`	`HOST_ACCESS_SEQUENTIAL_WRITE_BIT` + `MAPPED_BIT`
Readback	`HOST_VISIBLE` + `HOST_CACHED`	`HOST_ACCESS_RANDOM_BIT`
Per-frame data with ReBAR	`DEVICE_LOCAL` + `HOST_VISIBLE`	`HOST_ACCESS_ALLOW_TRANSFER_INSTEAD_BIT`
Render target, big or aliased	`DEVICE_LOCAL` (`LAZILY_ALLOCATED` on tilers)	`DEDICATED_MEMORY_BIT`

Sub-allocate: the number of `vkAllocateMemory` calls is limited (`maxMemoryAllocationCount`). Flush non-coherent ranges after writes.

Descriptor options

Option	Use
Sets and pools	`vkAllocateDescriptorSets`, `vkCmdBindDescriptorSets2`; layout by update frequency
Dynamic offsets	one big uniform or storage buffer, offset per draw
Push descriptors	core 1.4, `vkCmdPushDescriptorSet`, no pool
Push constants	`vkCmdPushConstants2`, at least 128 bytes
Descriptor indexing	1.2, update-after-bind, runtime arrays, `nonuniformEXT`
Buffer device address	1.2, pointers in shaders
Descriptor buffer	`VK_EXT_descriptor_buffer`
Descriptor heap	`VK_EXT_descriptor_heap`, Jan 2026, the announced successor

Barrier recipe (sync2)

```
VkImageMemoryBarrier2 b{
  srcStageMask, srcAccessMask,
  dstStageMask, dstAccessMask,
  oldLayout, newLayout,
  image, subresourceRange };
VkDependencyInfo d{
  imageMemoryBarrierCount = 1,
  pImageMemoryBarriers = &b };
vkCmdPipelineBarrier2(cmd, &d);
```

Buffers use `VkBufferMemoryBarrier2`, global dependencies `VkMemoryBarrier2`. An `UNDEFINED` old layout discards the contents.

Shader toolchain commands

```
glslc --target-env=vulkan1.3 -O -o a.spv
a.frag
dxc -spirv -T ps_6_0 -E main -fspv-target-
env=vulkan1.3 a.hlsl -Fo a.spv
slangc a.slang -target spirv -o a.spv
spirv-val --target-env vulkan1.3 a.spv
spirv-opt -O a.spv -o b.spv
spirv-dis a.spv
```

GLSL, HLSL and Slang all produce SPIR-V (1.6 with Vulkan 1.3+). Specialisation constants select variants at pipeline creation. Slang is the language of the Khronos tutorial.

Versions and promoted features

Version	Promoted or mandatory
1.1 2018	Features2 and Properties2, subgroup operations, multiview, 16-bit storage, maintenance1 to 3
1.2 2020	timeline semaphores, descriptor indexing, buffer device address, 8-bit storage, scalar layouts, SPIR-V 1.5
1.3 2022	dynamic rendering, synchronization2, `copy_commands2`, maintenance4, extended dynamic state 1 and 2, inline uniform blocks, SPIR-V 1.6, memory model
1.4 2024	local read, push descriptors, maintenance5 and 6, `map_memory2`, host image copy, line rasterisation, uint8 indices, vertex attribute divisor, pipeline robustness
Extensions	ray tracing, `VK_EXT_mesh_shader`, Vulkan Video, shader objects, descriptor heap

Legacy to modern

Legacy	Modern
`VkRenderPass`, `VkFramebuffer`	dynamic rendering
`vkCmdPipelineBarrier`, `vkQueueSubmit`	sync2: `...Barrier2`, `vkQueueSubmit2`
binary semaphores, fences	timeline semaphores
`vkGetPhysicalDeviceFeatures`	Features2, `Vulkan11` to `Vulkan14` structs
`VK_MAKE_VERSION`	`VK_MAKE_API_VERSION`
`VK_LAYER_LUNARG_*`, device layers	`VK_LAYER_KHRONOS_validation`
one allocation per resource	VMA sub-allocation
descriptor sets, descriptor buffer	descriptor heap (slated)
geometry shaders	mesh shaders or compute

Validation and debugging

- Enable `VK_LAYER_KHRONOS_validation`; configure with `vkconfig`, `VK_LOADER_LAYERS_ENABLE` or `VK_EXT_layer_settings`
- Modes: core, synchronization, GPU-assisted, Debug Printf (`debugPrintfEXT`), best practices
- `VK_EXT_debug_utils`: messenger, `vkSetDebugUtilsObjectNameEXT`, `vkCmdBeginDebugUtilsLabelEXT`
- `vulkaninfo --summary`, `vkcube`; API dump layer
- Capture: RenderDoc, GFXReconstruct; profile: Nsight, RGP, AGI
- Device lost: capture, then check the last submit and its barriers

Platform notes

- **Android:** NDK; set `preTransform` and rotate the clip matrix (pre-rotation); Swappy frame pacing; ANGLE optional from 15; Android Baseline profile
- **MoltenVK:** portability layer; enable `VK_KHR_portability_enumeration` and `ENUMERATE_PORTABILITY_BIT_KHR`; enable `VK_KHR_portability_subset` on the device
- **KosmicKrisp:** Vulkan 1.4 conformant on Metal 4, macOS 26+, in SDK 1.4.363
- **Surfaces:** Win32, Wayland, Xcb, Xlib, Android, Metal

Draw, dispatch and trace commands

Command	Use
`vkCmdDraw`, `vkCmdDrawIndexed`	direct draws, instancing
`vkCmdDrawIndexedIndirectCount`	GPU-driven draws (core 1.2)
`vkCmdDrawMeshTasksEXT`	task and mesh pipeline
`vkCmdDispatch`, `vkCmdDispatchIndirect`	compute, workgroups
`vkCmdTraceRaysKHR`	ray-tracing pipeline, shader binding table

Key URLs

- `vulkan.org`, `docs.vulkan.org/spec/latest`
- `docs.vulkan.org/guide/latest`
- `docs.vulkan.org/tutorial/latest`
- `registry.khronos.org/vulkan`
- `vulkan.lunarg.com/doc/sdk`
- `shader-slang.org`, `gpuopen.com/vulkan-memory-allocator`